Day 135: GAN Training Instability: a Real, Honest Mode-Collapse Demonstration

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 3 | Complete study guide

Watch: Ali Ghodsi, Deep Learning, GAN, Generative adversarial networks, AAE, Fall 2023, Lecture 16 | GAN Challenges Explained: Mode Collapse, Instability & WGAN Solutions (CodeLucky)

Script: `day135_gan_instability.py` · every number in this guide is copied by program from `day135_run_output.txt`, the real output of that script.

Mode collapse measured where every mode is known: eight tight Gaussians on a circle, with Day 134's GAN watched every 250 steps. With both players learning at the same rate, the usual non-saturating loss covered all eight modes in only 1 of 5 seeds, and in the other four the count at 3,000 steps was already the final count. The minimax loss, which textbooks warn against, covered all eight in 5 of 5. The textbook's reason shows up once the discriminator gets a 1,000-step head start (it then gives fakes 0.022 to 0.049): the non-saturating generator collapses onto 3 modes in three seeds, and the minimax generator's samples fall apart (good share down to 0.06). A discriminator that learns ten times slower than the generator is worst of all: good share 0.07 to 0.08.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

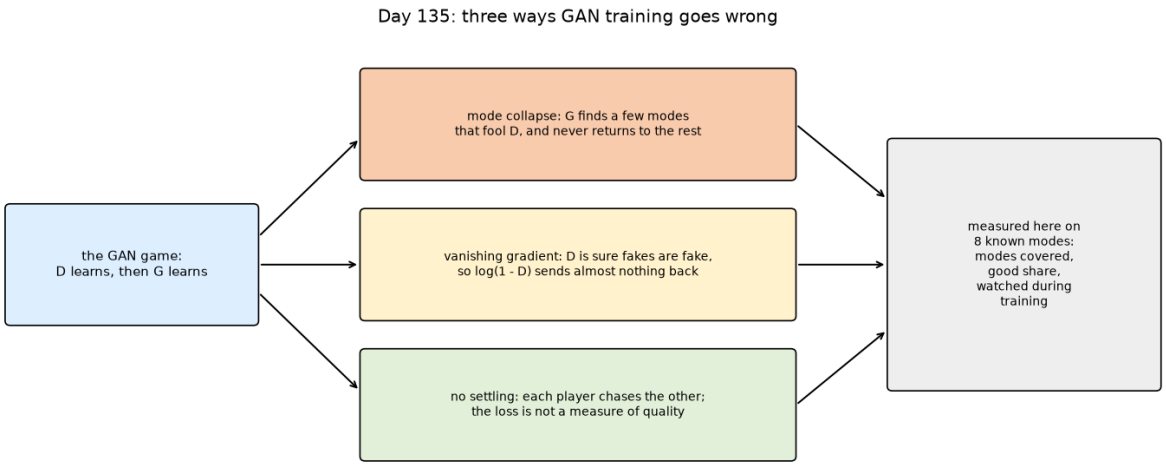
Mode collapse: the generator produces only part of the variety in the data. A **mode** is one peak of the data's distribution: one of the eight blobs here, one digit on MNIST. Day 134 saw it twice: 1-D runs that sat on one peak, and an MNIST generator that made a few near-identical images.

On real images, collapse is hard to measure. On a ring of eight Gaussians it is easy: count how many generated samples land near each centre (a **good** sample is within 3 spreads of one), and say a mode is **covered** if it gets at least 1% of them.

REAL-WORLD SCENARIO

A street musician who finds that one song earns coins and plays only that song. Every performance is good; the repertoire is gone.

SYSTEM DIAGRAM



Three ways GAN training goes wrong, each measured here on eight known modes.

Today's part for the week's build

`day135_exercise.py` is the ring, the mode count and the watched training loop, checked by hand (the good share of real samples against $1 - e^{-4.5}$). Its last line is seed 0 again for 2,000 steps: 7 modes at step 2,000, where section 2's run of the same seed has 6. A likely reason, not tested here: the exercise runs with OpenBLAS's default number of threads and the lesson with 4, which changes the order of floating-point sums, and a GAN amplifies tiny differences.

ACTUAL OUTPUT

```
100,000 real samples: 8 of 8 modes covered; good share 0.9891; by hand 1 - e^(-4.5) = 0.9889
radius of the real samples: average 2.0007 (centres at 2.0)
3,000 samples from modes 0, 1 and 2 only: 3 modes covered; counts [1006, 986, 976, 0, 0, 0, 0, 0]
a 2,000-step GAN, watched: step 500: 1 modes, good 0.01; step 1000: 0 modes, good 0.01; step 1500:
    4 modes, good 0.12; step 2000: 7 modes, good 0.39
```

Data: generated by the script (8 Gaussians of spread 0.05 on a circle of radius 2). Day 134's gan_step, generate and g_loss and Day 126's networks and Adam are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: the gradient a fake sends back

DEFINITION — start here, no prior knowledge assumed

Day 134 showed one case: when D is sure a fake is fake, the minimax loss log(1 – D) sends back almost nothing. The table repeats it at four levels of D(fake). Per fake, the non-saturating gradient (with respect to D's logit) is D – 1 and the minimax gradient is –D.

REAL-WORLD SCENARIO

Two coaches: one says 'try harder' most loudly when you are losing badly; the other goes quiet exactly then.

MATHEMATICS

$$d/dl\ [-\log D] = D - 1, \quad d/dl\ [\log(1 - D)] = -D, \quad \text{ratio} = (1 - D) / D$$

IMPLEMENTATION

```
def demo_hand():
    print("=" * 70)
    print("1. BY HAND -- the gradient one fake sends back to the generator, under each loss")
    print("=" * 70)
    print("   D(fake)   non-saturating -log D   minimax log(1 - D)   ratio")
    rows = []
    for p in (0.5, 0.1, 0.01, 0.001):
        lf = np.full((1, 1), np.log(p / (1 - p)), np.float32)
        a, b = float(d134.g_loss(lf)[1][0, 0]), float(d134.g_loss(lf, "minimax")[1][0, 0])
        rows.append((p, a, b)); print(f"   {p:7.3f}   {a:+21.4f}   {b:+18.4f}   {a / b:5.0f}")
    print("by hand: non-saturating d/dlogit = D - 1; minimax = -D. When D(fake) is near 0 the
        minimax gradient is near 0 too")
    print("Actual output from this lesson's run.")
    return rows
```

ACTUAL OUTPUT

```
   D(fake)   non-saturating -log D   minimax log(1 - D)   ratio
     0.500                -0.5000              -0.5000       1
     0.100                -0.9000              -0.1000       9
     0.010                -0.9900              -0.0100      99
     0.001                -0.9990              -0.0010     999
by hand: non-saturating d/dlogit = D - 1; minimax = -D. When D(fake) is near 0 the minimax
    gradient is near 0 too
Actual output from this lesson's run.
```

What it means

At D(fake) = 0.5 the two gradients are equal; at 0.001 the non-saturating one is 999 times larger. That is the textbook argument for the non-saturating loss. Sections 3 and 4 test when it actually matters.

WHY CHOOSE THIS?

The whole argument for the non-saturating loss, in one formula.

WHY NOT THIS? (TRADEOFFS)

It says nothing about which modes the generator covers.

2. Eight peaks, five seeds: how often is a mode lost?

DEFINITION — start here, no prior knowledge assumed

Day 134's GAN (non-saturating loss, Day 126's Adam at 0.001 for both players) trained five times on the ring. Every 250 steps, the same 2,000 noise vectors go through the generator, and the script counts modes covered and the share of good samples. **Lost** counts the times a covered mode dropped below 1% between two looks; **found**, the times one appeared (including the first time).

REAL-WORLD SCENARIO

Counting which of eight dishes a cook still makes, every week, rather than tasting one plate at the end.

SPECIFICATION

Data	8 Gaussians, spread 0.05, on a circle of radius 2; batches of 256
Players	G [2, 128, 128, 2] (noise of length 2), D [2, 128, 128, 1]; 6,000 steps
Measured	every 250 steps on fixed noise: modes with at least 20 good samples of 2,000, good share, D(fake)

IMPLEMENTATION

```
def mode_stats(F, k=8, radius=2.0, sd=0.05):
    C = np.c_[np.cos(2 * np.pi * np.arange(k) / k), np.sin(2 * np.pi * np.arange(k) / k)] * radius
    D = np.sqrt(((F[:, None, :] - C[None]) ** 2).sum(-1)); near = D.argmin(1)
    good = D[np.arange(len(F)), near] < 3 * sd
    counts = np.bincount(near[good], minlength=k)
    return int((counts >= 0.01 * len(F)).sum()), float(good.mean()), counts
```

ACTUAL OUTPUT

```
non-saturating loss, -log D(G(z)): modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       1     1     6     7     7     8     8   0.89     0.44     4     12  165 268  34 263 329 301  91 334
      1       0     0     5     7     7     7     7   0.91     0.47     3     10  262 293 242 293   0 201 198 333
      2       0     5     6     7     7     7     7   0.95     0.47     1      8  238   0 457 229 292 176 248 254
      3       0     0     5     6     6     6     6   0.94     0.44     3      9    0 302 338 331   1 312 300 290
      4       1     2     5     5     5     5     5   0.91     0.37     1      6  403   0 338 286   0 403 396   0
   all 8 modes at the end in 1 of 5 runs (161 s)
Actual output from this lesson's run.
```

What it means

Only seed 0 ended with all eight modes. The others ended with 7, 7, 6 and 5, and the failure is coverage, not quality: their good shares are 0.91 to 0.95. The missing modes stayed missing: seeds 1 to 4 had the same count at 3,000 steps (seed 4 from 2,000) as at 6,000; only seed 0 gained a mode after 3,000 steps. The mode-by-mode panel of the worst run (seed 4) shows its three missing modes at zero while the five it found share the samples.

WHY CHOOSE THIS?

Mode coverage and sample quality are two separate numbers, each measurable here.

WHY NOT THIS? (TRADEOFFS)

Eight blobs in 2-D; real data has no list of its modes.

3. The minimax loss on the same five seeds

DEFINITION — start here, no prior knowledge assumed

The only change: the generator minimises log(1 – D(G(z))), the original formula, instead of –log D(G(z)).

REAL-WORLD SCENARIO

The same cook, given a different kind of feedback, on the same eight dishes.

ACTUAL OUTPUT

```
minimax loss, log(1 - D(G(z))): modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       0     0     8     8     8     8     8   0.89     0.47     1      9  181 235 274 266 214 192 217 197
      1       0     3     6     8     7     8     8   0.88     0.43     3     11  236 111 189 162 197 285 319 259
      2       0     1     8     8     8     8     8   0.91     0.50     2     10  193 212 262 240 253 207 234 220
      3       0     0     8     8     8     8     8   0.85     0.47     0      8  218 256 175 281 179 148 184 255
      4       0     4     8     8     8     8     8   0.89     0.49     3     11  227 245 217 241 242 219 182 213
   all 8 modes at the end in 5 of 5 runs (158 s)
Actual output from this lesson's run.
```

What it means

The minimax loss covered all eight modes in 5 of 5 runs, against 1 of 5 for the non-saturating loss. This is the opposite of the usual advice, and it is what this setting measured. The section 1 argument only bites when D is confident, and here it never was: D(fake) ended between 0.43 and 0.50. Why the minimax loss also covered more modes was not tested; five seeds of one toy problem do not make it the better loss in general.

WHY CHOOSE THIS?

Covered every mode here, with equal learning rates.

WHY NOT THIS? (TRADEOFFS)

Fails badly once the discriminator is confident (section 4).

4. A discriminator with a head start

DEFINITION — start here, no prior knowledge assumed

For the first 1,000 steps the generator's learning rate is 0, so only the discriminator learns. When the generator starts, D is already confident its fakes are fake: the situation the section 1 argument is about. Both losses, five seeds each.

REAL-WORLD SCENARIO

A new forger facing an inspector who has already studied a thousand of the forger's clumsy early fakes.

IMPLEMENTATION

```
def train_ring(seed, steps=6000, kind="non-saturating", lr_g=1e-3, lr_d=1e-3, d_steps=1,
    every=250, k=8, z=2, head=0):
    rng = np.random.RandomState(seed); G = d126.init_net([z, 128, 128, 2], seed); D =
        d126.init_net([2, 128, 128, 1], seed + 100)
    oG, oD = d126.Adam(G, lr_g), d126.Adam(D, lr_d); track = []
    ev = np.random.RandomState(999).standard_normal((2000, z)).astype(np.float32)
    for t in range(1, steps + 1):
        oG.lr = 0.0 if t <= head else lr_g
        _, _, dr, df = d134.gan_step(G, D, oG, oD, ring(256, rng, k), rng, z, out="linear",
            kind=kind, d_steps=d_steps)
        if t % every == 0:
            m, g, c = mode_stats(d134.generate(G, ev, "linear")[0], k); track.append((t, m, g, c,
                dr, df))
    return G, track
```

ACTUAL OUTPUT

```
for 1,000 steps only the discriminator learns, so by the time the generator starts it is sure the fakes are fake
non-saturating loss, head start 1,000 steps: modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       0     0     4     5     6     6     6   0.89     0.41     2      8  343 247 315 153   1 379 351   0
      1       0     0     3     3     3     3     3   0.94     0.27     0      3    0   0   0   0   0 455 691 727
      2       1     1     5     8     8     8     8   0.92     0.49     2      9  427 458 133 149 137 140 138 251
      3       0     0     3     3     3     3     3   0.87     0.22     0      3  622 524   0   0   0   0   0 602
      4       0     0     2     5     3     3     3   0.91     0.30     4      7    5 446   4   2   0   0 606 762
   all 8 modes at the end in 1 of 5 runs (173 s)
   D(fake) at step 1,000, when the generator starts: 0.022 0.023 0.045 0.049 0.043
minimax loss, head start 1,000 steps: modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       0     0     3     3     4     6     6   0.45     0.23    13     19   80 144  13   5  27 285 320  23
      1       0     0     3     4     6     7     6   0.51     0.29    12     18  236 170   8  68  64  18 201 249
      2       1     1     1     3     3     5     1   0.06     0.07    25     25    9  19   7   1  78   1   1   0
      3       0     0     3     3     3     3     3   0.92     0.22     0      3  622 577   0   0   0   0   0 648
      4       0     0     1     1     0     2     4   0.14     0.16    10     14   31 174  29   2  12   6   3  24
   all 8 modes at the end in 0 of 5 runs (172 s)
   D(fake) at step 1,000, when the generator starts: 0.022 0.023 0.045 0.049 0.043
Actual output from this lesson's run.
```

What it means

When the generator starts, D gives fakes 0.022 to 0.049. The minimax generator, whose gradient is about D(fake), never organises: good shares of 0.45, 0.51, 0.06 and 0.14 in four seeds, and modes lost 13, 12, 25 and 10 times, jumping from blob to blob. The non-saturating generator does learn, and its samples stay good (0.87 to 0.94), but it collapses: three of five seeds end on just 3 modes. Seed 3 collapsed the same way under both losses. A strong discriminator early on hurts both losses, in different ways.

WHY CHOOSE THIS?

Shows the vanishing gradient directly, and that the non-saturating fix does not prevent collapse.

WHY NOT THIS? (TRADEOFFS)

A head start is an extreme way to make D confident; real training drifts there more gradually.

5. The balance between the players

DEFINITION — start here, no prior knowledge assumed

Three changes to the balance, non-saturating loss, seeds 0 to 2: the generator learning ten times slower than the discriminator, the discriminator ten times slower, and five discriminator steps for every generator step.

REAL-WORLD SCENARIO

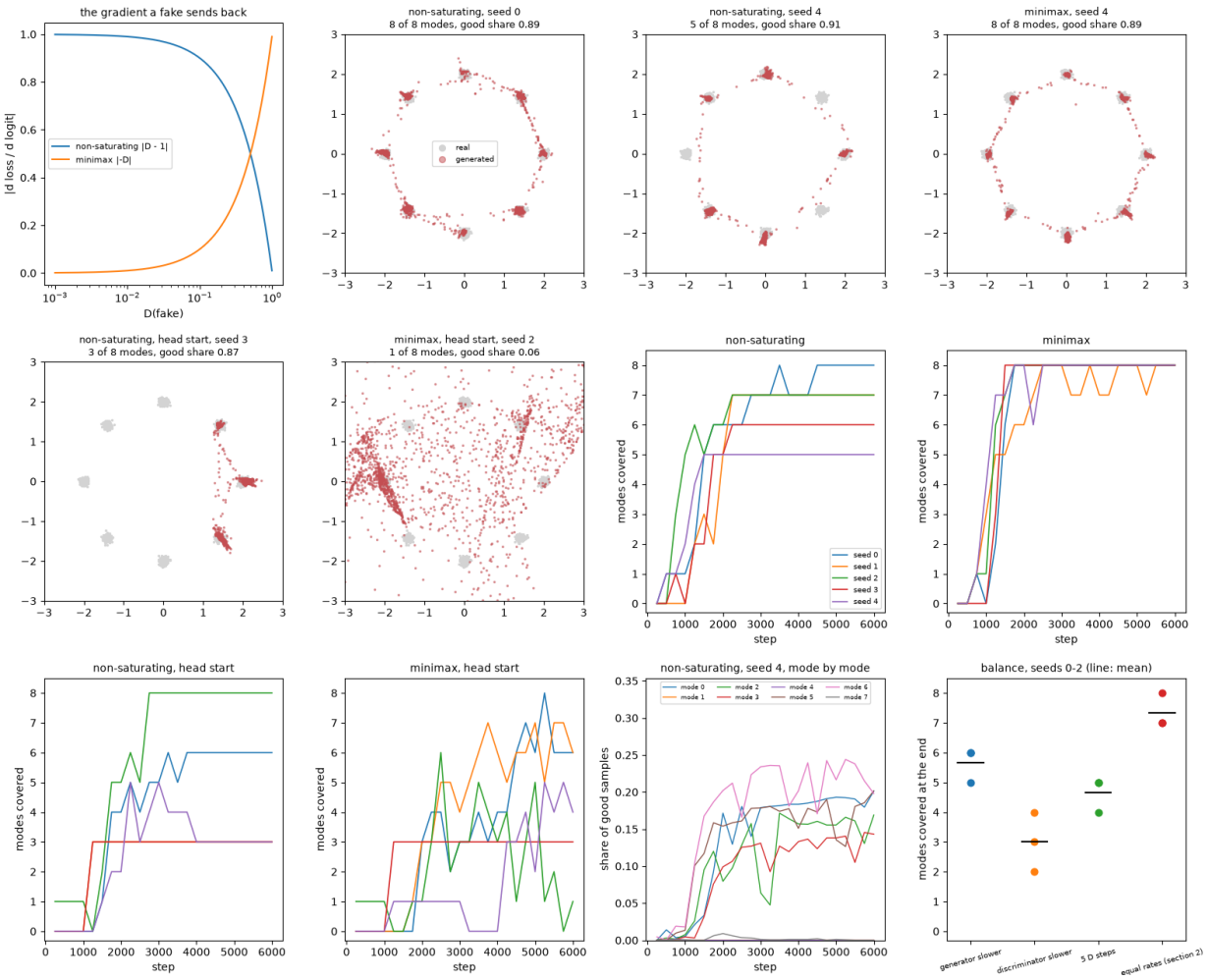
A tennis match between a beginner and a champion teaches neither of them much.

ACTUAL OUTPUT

```
generator slower: G 0.0001, D 0.001: modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       3     4     6     6     6     6     6   0.82     0.39     1      7  164 386   0 327 328 197 218  16
      1       3     4     4     4     5     5     5   0.78     0.34     0      3    9 315   1  18 301 528 288  90
      2       2     5     6     6     6     6     6   0.84     0.37     1      5  190 402   2 234   0 296 301 265
   all 8 modes at the end in 0 of 3 runs (101 s)
discriminator slower: G 0.001, D 0.0001: modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       0     0     1     1     3     2     3   0.07     0.42    26     29   36   1  17  17  26   0  12  24
      1       1     1     1     3     3     3     4   0.08     0.34    38     41    0   0   0  63  34  21  33   0
      2       1     1     2     2     2     1     2   0.08     0.47    31     33   16  17  35  16  19  15  15  21
   all 8 modes at the end in 0 of 3 runs (83 s)
5 discriminator steps per generator step: modes covered (of 8) at each step, then good share and D(fake) at the end
   seed     500 1,000 2,000 3,000 4,000 5,000 6,000   good  D(fake)  lost  found  final counts per mode (of 2,000)
      0       3     5     6     5     6     5     5   0.71     0.35     6      9    0 276 129   2 372 179 455  12
      1       3     4     5     5     6     5     5   0.34     0.23    11     16    0  53  39   3 174  61 349   4
      2       2     4     3     3     4     3     4   0.21     0.22    18     20   55   9 105 178   0  71  11   0
   all 8 modes at the end in 0 of 3 runs (306 s)
Actual output from this lesson's run.
```

What it means

None of the nine runs covered all eight modes; with equal rates, seed 0 did (section 2). A slow discriminator was worst: good share 0.07 to 0.08, with modes lost 26 to 38 times; a likely reason, not tested here, is that a critic that barely learns gives the generator nothing steady to aim at. Five discriminator steps (a common recipe for other GAN losses) gave good shares of 0.71, 0.34 and 0.21. A slow generator kept quality (0.78 to 0.84) but found only 5 or 6 modes.



Top: the two gradients against D(fake); the best and worst non-saturating runs and the minimax run of the same worst seed. Middle: the worst head-start runs; modes covered over training for every seed. Bottom: the same with a head start; one run mode by mode; final modes covered for each balance setting.

WHY CHOOSE THIS?

Equal learning rates were the best of the balances tried.

WHY NOT THIS? (TRADEOFFS)

Three seeds per setting; a finer search of learning rates was not run.

Interview preparation — technical questions

Q: How would you measure mode collapse?

A: On data with known modes: count samples near each mode (covered if at least 1%) and the share of good samples, during training.

Q: Why is the non-saturating loss usually preferred?

A: Its gradient per fake is D – 1, not –D: 999 times larger at D(fake) = 0.001.

Q: Did the non-saturating loss always win here?

A: No: with equal learning rates the minimax loss covered all 8 modes in 5 of 5 seeds, the non-saturating loss in 1 of 5.

Q: What happens when the discriminator is too strong early?

A: Minimax: good share down to 0.06 and constant mode hopping; non-saturating: collapse onto 3 modes in three seeds.

Q: Why watch a GAN during training?

A: Coverage changes over time, and its losses do not say how many modes are covered.

Interview preparation — theoretical questions

Q: What is mode collapse?

A: The generator covers only some modes of the data, often with good samples on those it does cover (good share 0.91 to 0.95 here).

Q: Why does a too-slow discriminator fail?

A: Good share was 0.07 to 0.08. A likely reason, not tested here: its verdicts barely change, so the generator gets no steady direction.

Q: Are GAN results reproducible?

A: Not across machines or settings: the same seed in the exercise gave 7 modes at step 2,000 instead of 6, likely from a different thread count.

Q: What fixes for collapse exist?

A: Not tested here: Wasserstein losses, minibatch discrimination and gradient penalties are the usual next steps.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Mode collapse is real, common and measurable. With the usual settings, four of five runs lost one to three modes for good while producing good samples on the rest. The textbook loss advice held only in the situation it was derived for (a confident discriminator), and every imbalance between the players made things worse. A GAN has to be watched during training, on a measure of coverage as well as quality.

- Non-saturating, equal rates: all 8 modes in 1 of 5 seeds; in the other four, the count at 3,000 steps was final.
- Minimax, equal rates: all 8 modes in 5 of 5 seeds.
- Head start (D(fake) 0.022 to 0.049): minimax good share down to 0.06; non-saturating collapsed to 3 modes in 3 seeds.
- Slow discriminator: good share 0.07 to 0.08.

Suggested watch order, if starting from zero

- 1. Ali Ghodsi, Deep Learning, GAN, Generative adversarial networks, AAE, Fall 2023, Lecture 16 — a university lecture on GANs, mode collapse and minibatch discrimination.
- 2. GAN Challenges Explained: Mode Collapse, Instability & WGAN Solutions (CodeLucky) — a short tour of mode collapse, instability and the Wasserstein fix.

Day 136 preview

- **A DCGAN-style convolutional GAN**: the same game with convolutional players, so the generator can draw strokes.